

ESG Data Platform - Technical Deep Dive

Technical Deep Dive — Full Build Log, Decisions & Reasoning

Ishan Peshkar

July 2026

Enterprise ESG Data Platform — Technical Deep Dive

1. Backstory & Motivation

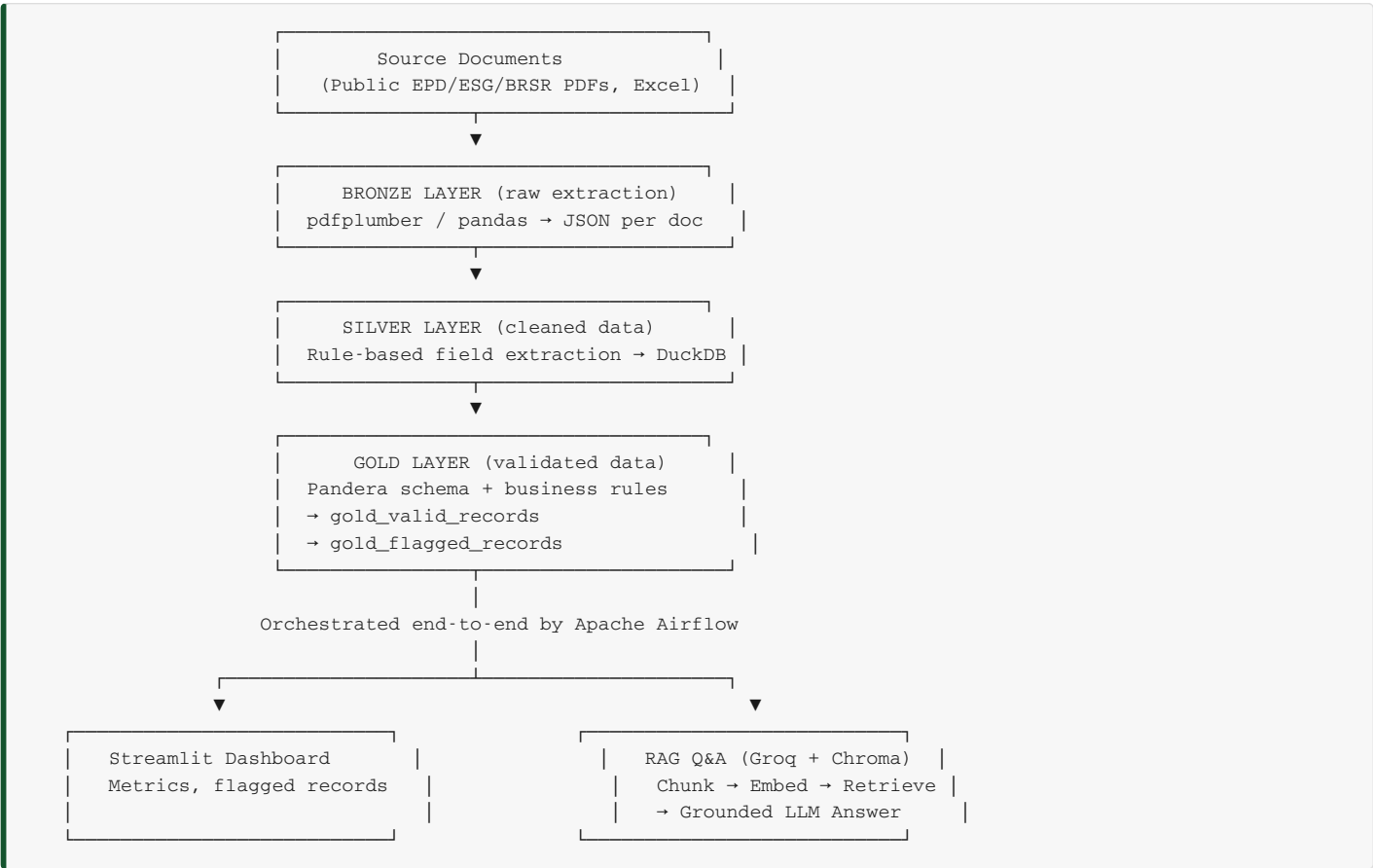
While interning on an ESG/EPD document analysis project — extracting data from sustainability documents and assessing it against regulatory rulesets — I wanted to build a parallel personal project to (a) gain hands-on Data Engineering skills beyond my ML/GenAI/RAG background, and (b) apply that Data Engineering skillset to a domain I already understood deeply from the internship, without using any real or confidential company data.

Goal: build an “Enterprise ESG Data Platform” — a scaled-down, personal, fully local version of the kind of ESG data platform companies like Deloitte, EY, and Infosys build for enterprise clients.

2. Constraints & Ground Rules

- No paid tools or cloud subscriptions (no AWS) — everything runs locally
- No real internship data used anywhere — only public EPD/ESG/BRSR documents
- Learning-first: build in phases, prove each layer works before adding complexity
- Domain reused from the internship (ESG/EPD assessment), but all code and infrastructure built independently from scratch

3. Final Architecture



4. Tech Stack Decisions

Layer	Tool	Why
Extraction	pdfplumber, pandas/openpyxl	Free, standard, no API cost
Storage	Local filesystem + DuckDB	Zero-setup analytical DB, mimics a real warehouse
Validation	Pandera + custom business rules	Lightweight schema validation, beginner-friendly vs. Great Expectations
Orchestration	Apache Airflow (Docker + WSL2)	Industry-standard, stronger resume signal than Prefect despite added setup complexity on Windows
Dashboard	Streamlit	Fast to build, directly queries DuckDB
Embeddings	sentence-transformers (all-MiniLM-L6-v2)	Free, local, ~80MB, well-established baseline
Vector store	ChromaDB	Free, local, no server required
LLM	Groq (Llama 3.3 70B)	Free tier, fast inference, strong open-weight model quality

Phase 0 — Foundations

Completed prior to formal logging. Learned: ETL vs. ELT, data lake vs. warehouse vs. lakehouse, Docker basics, and the medallion (bronze/silver/gold) architecture pattern.

Phase 1 — Manual Pipeline (No Orchestration)

Step 1.1 — Environment Setup

- Created project folder structure (`data/` , `src/extract` , `src/clean` , etc.)
- Set up a Python virtual environment
- Installed: `pdfplumber` , `pymupdf` , `pandas` , `openpyxl` , `duckdb` , `python-dotenv`

Step 1.2 — Source Documents

- Collected 8 public EPD/ESG/BRSR PDFs (ABB, Lival OY, ecoinvent, PolyCo, Tata Steel, a BRSR report, a GRI report, a network cable EPD)
- **Decision:** use only public documents, never real internship files

Step 1.3 — Extraction (Bronze Layer)

- Built `pdf_extractor.py` — extracts raw text + tables per page via pdfplumber, saved as JSON in `data/bronze/pdf/`
- Built `excel_extractor.py` — extracts all sheets/rows via pandas, saved as JSON in `data/bronze/excel/`
- **Key decision:** bronze layer = raw/untouched data, no cleaning applied yet

Step 1.4 — Cleaning (Silver Layer)

Excel cleaner: - Attempt 1 failed: `InvalidInputException: Could not convert string 'Tin' to INT32` — real-world ESG Excel templates (e.g. Conflict Minerals Reporting Template) mix text and numbers in the same column due to multi-row headers/section labels. - Attempt 2 failed: `ValueError: The truth value of a Series is ambiguous` — duplicate column names (multiple “Unnamed: X” columns) caused `df[col]` to return a DataFrame instead of a Series. - **Decision:** parked Excel silver-layer cleaning. Bronze extraction for Excel had already proven the core concept; deeper cleaning of messy multi-header templates was deprioritized as a rabbit hole not central to the learning goals.

PDF cleaner: - Built naive rule-based (regex) field extraction for: `product_name` , `declared_unit` , `reporting_period` ,

`gwp_total`, `standard_reference` - First run: `standard_reference` worked reasonably well (caught EN 15804 variants). Other fields were inconsistent — `product_name` grabbed section headings or mid-sentence fragments; `gwp_total` was almost entirely null. - **Iteration attempt:** restricted header-style fields to the first 3 pages, added length caps to reject clearly-wrong matches, widened `gwp_total` patterns. - **Bug hit:** used `^ / $` line anchors without the `re.MULTILINE` flag, causing `product_name` to match nothing at all. Fixed by adding `re.MULTILINE`. - **Result after fix:** `gwp_total` and `reporting_period` improved meaningfully; `product_name` remained unreliable — different issuers either omit an explicit label entirely or use inconsistent phrasing/placement, which regex cannot resolve without contextual understanding. - **Decision:** stopped regex tuning at this point — diminishing returns. This produced concrete, reproducible evidence that keyword/regex extraction breaks down on real-world document variance, directly motivating the move to LLM-based extraction used later in Phase 5's RAG system.

Phase 1 status: COMPLETE.

Phase 2 — Data Modeling + Validation

Step 2.1 — Define Validation Schema

- Installed `pandera` (hit a transient PyPI 503 error on first attempt; resolved on retry)
- Built a `DataFrameSchema` for `silver_pdf_esg_data`: hard requirements on `_source_file`, `page_count`, `char_count`; nullable/conditional checks on `product_name` (max length), `gwp_total` (numeric-pattern regex), and `standard_reference` (must match known standards)
- Decision:** validated `gwp_total` as a string with a numeric regex pattern rather than casting to float, to avoid type-casting crashes (a lesson carried over from the Excel cleaning failures)

Step 2.2 — Run Validation

- Built a validation script loading silver data and running it against the schema with `lazy=True` (collects all failures, not just the first)
- Bug hit:** `ModuleNotFoundError` when running as a module — fixed by adding `__init__.py` files and using absolute imports
- Result:** all 8 rows passed validation
- Cleanup:** updated imports to `pandera.pandas` to resolve a deprecation warning

Key insight: all rows passing validation did *not* mean the data was correct — it meant the data was *structurally well-formed*. Values like `gwp_total = 16` passed because they were valid numbers, despite strong evidence they were mis-extracted footnote digits rather than real GWP values. This demonstrated the gap between **syntactic validation** (is it shaped correctly) and **semantic validation** (does it mean what it should) — a gap that rule-based schemas alone cannot close.

Step 2.3 — Business Rules + Gold Layer Split

- Built domain-specific business rules derived directly from observed Phase 1 failures: a `product_name` blocklist (certification, management system, declaration number, etc.) and a “suspiciously small GWP” threshold to flag likely mis-extracted digits
- Built a script applying both schema validation and business rules, splitting output into `gold_valid_records` and `gold_flagged_records` DuckDB tables
- Result:** 0 clean records, 8/8 flagged. This was the correct, expected outcome — the business rules were built directly from known Phase 1 weaknesses, so the validation layer correctly surfaced that 100% of the current extraction was semantically unreliable. A validation layer's job is to honestly expose where data is *not* trustworthy, not to make the data look clean.

Decision point: with the pipeline now proven end-to-end but limited by weak extraction, considered whether to fix extraction immediately or proceed to orchestration. **Chose to proceed to orchestration** — Airflow/DAG concepts apply regardless of data quality, and rebuilding extraction first would have required retrofitting orchestration around a changed pipeline shape later. Improving extraction was deferred to Phase 5 as originally planned (LLM-based extraction via RAG).

Phase 2 status: COMPLETE.

Phase 3 — Orchestration (Apache Airflow)

Decision: Prefect vs. Airflow

Initially considered Prefect for its simplicity (pure Python, native Windows support, no Docker required). **Reversed the decision in favor of Airflow**, since it is more commonly required in Data Engineering job listings — accepting the added setup complexity

(Docker + WSL2 on Windows) as a worthwhile tradeoff for stronger industry-relevance.

Step 3.1 — WSL2 + Docker Setup

- Updated WSL2, set default version to 2, installed Docker Desktop with WSL2 engine integration
- Verified with `docker --version`, `docker run hello-world`, and `docker compose version` — all succeeded on first attempt

Step 3.2 — Airflow Setup via Docker Compose

- Created `airflow/` folder structure (`dags/` , `logs/` , `plugins/` , `config/`)
- Downloaded the official Airflow `docker-compose.yaml` (v2.10.4)
- Created `.env` with `AIRFLOW_UID=50000` (standard Windows workaround, since Linux user-ID auto-detection doesn't apply)
- Ran `docker compose up airflow-init` then `docker compose up -d`
- Verified Airflow UI accessible at `http://localhost:8080`

Step 3.3 — Connect Airflow to Project Code

- Added a volume mount (`../:/opt/airflow/esg_project`) so Airflow containers could see the full project directory
- Set `_PIP_ADDITIONAL_REQUIREMENTS` so containers had the same dependencies as the local virtual environment
- Verified the mount via `docker exec ... ls /opt/airflow/esg_project`

Step 3.4 — Build the DAG

- Built `esg_pipeline_dag.py` using `BashOperator` tasks wired in sequence: `extract_pdf >> clean_pdf >> validate_pdf >> build_gold_layer`
- DAG appeared correctly in the Airflow UI after being placed in `dags/`

Step 3.5 — First DAG Run & Idempotency-Adjacent Bug

- First trigger: `extract_pdf` succeeded, `clean_pdf` failed with `ModuleNotFoundError: No module named 'duckdbQ'` — a stray typo (`import duckdbQ` instead of `import duckdb`) in a local file, surfaced by Airflow executing the script fresh. Downstream tasks correctly showed `upstream_failed`, demonstrating Airflow's dependency enforcement working as intended.
- Fixed the typo, re-triggered the DAG — all 4 tasks completed successfully in order, output verified to match the known-correct manual run (0 valid, 8 flagged records).

Incident — Docker Desktop Container/Volume Wipe

Mid-project, Docker Desktop was found with zero containers and zero volumes (root cause unclear — suspected update/reset). Project logic was not lost, since `docker-compose.yaml`, `.env`, and the DAG file all live on the filesystem, not inside Docker's internal storage. Recovered by re-running `docker compose up airflow-init` (re-pulling ~400MB of images) and `docker compose up -d`. This distinction — files on disk vs. state inside Docker's internal storage — was a useful practical lesson in how Docker Compose actually persists (or doesn't persist) different kinds of data.

Phase 3 status: COMPLETE.

Phase 4 — Dashboard (Streamlit)

Step 4.1 — Install Streamlit

Installed cleanly; `streamlit hello` confirmed the install via its built-in demo app.

Step 4.2 — Build Dashboard + Idempotency Bug

- Built `src/dashboard/app.py`, reading `gold_valid_records` and `gold_flagged_records` from DuckDB, showing summary metrics (total/valid/flagged/pass rate) and tabbed views with a flag-reason filter
- **Bug discovered:** dashboard showed 24 total documents instead of the expected 8. **Root cause:** `pdf_extractor.py` saved bronze JSON files with a timestamp in the filename, so every pipeline run (including repeated Airflow triggers) created *new* files instead of overwriting existing ones — silver/gold layers then processed duplicated bronze records across multiple runs.
- **Concept:** this was a violation of **idempotency** — re-running the pipeline did not produce the same result as running it once. This is a significant, realistic Data Engineering issue: in a scheduled/production pipeline, bronze data would grow

unbounded with duplicates over time.

- **Fix:** removed the timestamp from the bronze filename (kept it as metadata *inside* the JSON content instead), so filenames are now based on source filename only — re-running now overwrites rather than duplicates. Cleared existing duplicates and re-ran the full pipeline to confirm the correct 8-document count returned.

Phase 4 status: COMPLETE.

Phase 5 — AI-Powered ESG Q&A (RAG)

Decision: LLM Provider

Considered a paid API (OpenAI/Anthropic) vs. fully local (Ollama) vs. Groq. **Chose Groq** — free tier, fast inference, hosts strong open-weight models (Llama family) via an OpenAI-compatible API, keeping the project fully cost-free while avoiding the hardware/setup overhead of running models locally via Ollama.

Step 5.1 — Groq API Setup

- Installed the Groq SDK, stored the API key in `.env` (excluded from git)
- Built a minimal test script confirming end-to-end connectivity — received a correct, coherent one-sentence definition of an EPD from `llama-3.3-70b-versatile`

Step 5.2 — Chunking + Embeddings (Vector Store)

- Installed `sentence-transformers` and `chromadb`
- **Bug hit:** `OSError: [WinError 206] The filename or extension is too long` while installing `torch` — caused by Windows' 260-character MAX_PATH limit being exceeded by deeply nested folders inside the torch package, compounded by an already-long project path.
- **Fix (no restart required):** used `subst X: "<project path>"` to map a short virtual drive letter to the project folder, shortening effective paths below the limit. Reinstalled successfully from the mapped drive.
- Built `build_vector_store.py`: chunks bronze PDF text (800 characters, 150-character overlap to preserve context across boundaries), embeds each chunk using `all-MiniLM-L6-v2` (free, local, ~80MB), and stores embeddings + text + metadata (source file, page number) in a persistent ChromaDB collection. The collection is deleted and recreated on each run for idempotency — the same lesson learned from the bronze/gold layer duplication bug.
- **Result:** 8 bronze PDF records produced 849 chunks, all successfully embedded and stored.

Step 5.3 — Retrieval + Grounded Generation

- Built `ask_esg.py`: embeds the user's question, retrieves the top-5 most similar chunks from ChromaDB, and constructs a prompt instructing the LLM to answer *only* from the retrieved context, citing sources, and to explicitly state when an answer isn't present rather than guessing.
- **Test query:** "What standards are referenced in these EPD documents?" **Result:** correctly identified EN 15804, the International EPD System General Programme Instructions v5.0, and PCR 2019:14 — all grounded in retrieved chunks with correct source file and page citations, synthesized across multiple documents in one coherent answer. This was a clear, demonstrable upgrade over the naive regex-based `standard_reference` field from Phase 1.

Step 5.4 — Interactive Q&A in Dashboard

- Added a "Ask a Question" tab to the Streamlit dashboard, reusing the same embedding model, ChromaDB collection, and Groq client (cached via `st.cache_resource`)
- **Test query:** "What is the relation between EPD and ESG, and how does BRSR fit into it?" **Result:** retrieved chunks from both an EPD document and the BRSR report (correct cross-source retrieval). The model explicitly noted that the context did not directly state the relationship, then reasoned carefully from what it *did* retrieve to construct an accurate, well-cited answer — demonstrating appropriate epistemic honesty rather than a fabricated but confident-sounding answer.

Step 5.5 — Edge Case Testing (Out-of-Scope Questions)

Tested deliberately irrelevant questions to verify the system doesn't hallucinate using the underlying LLM's general training knowledge: - "What is the carbon footprint of Apple's iPhone 15 supply chain?" → correctly stated Apple/iPhone 15 was not present in the retrieved context (which covered TCS's report and an unrelated LCA document) - "What is the current stock price of Tesla?" → correctly stated Tesla was not present in the retrieved context

Key insight: the vector retriever always returns its top-K nearest chunks regardless of true relevance — cosine similarity search has no built-in “nothing matches” threshold. The critical safeguard is the *generation* step: an explicit prompt instruction to answer only from provided context, which correctly overrode the LLM’s general pretrained knowledge in both tests. This confirms the grounding discipline built into the system works even under adversarial/out-of-scope questioning.

Phase 5 status: COMPLETE.

6. Summary of Engineering Lessons

1. **Bronze/silver/gold separation** made every failure traceable to a specific layer rather than a tangled mess of “the pipeline is broken somewhere.”
2. **Idempotency is not automatic** — it must be deliberately designed for (keying outputs by content identity, not by run timestamp), and its absence causes silent, compounding data quality problems rather than loud failures.
3. **Validation systems should expose distrust, not manufacture trust.** A 100% flag rate was a successful test of the validation logic, not a failed project outcome.
4. **Regex/keyword extraction has a real, demonstrable ceiling** on real-world document variance — a ceiling that motivates (rather than merely asserts) the case for LLM-based, context-aware extraction.
5. **RAG grounding discipline must be explicit.** Retrieval alone does not prevent hallucination; the generation prompt must explicitly instruct the model to defer to retrieved context and to admit gaps.
6. **Infrastructure incidents (Docker wipes, Windows path limits, PyPI outages) are part of real Data Engineering work**, not distractions from it — diagnosing and resolving them was as instructive as the “planned” parts of the project.

7. Possible Future Extensions

- Replace regex-based PDF field extraction with LLM-based structured extraction (the gap explicitly identified and validated throughout Phases 1, 2, and 5)
- Revisit Excel silver-layer cleaning with proper multi-header parsing
- Add DAG scheduling (e.g. daily) and failure alerting in Airflow
- Expand source coverage (additional public EPD/ESG document types, government APIs)
- Investigate and address LLM extraction non-determinism (a related, separately-explored problem: repeated extraction runs on the same document producing different field counts)